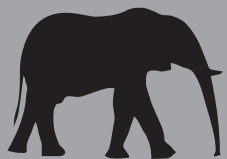


Git à 100%

mémento

Raphaël Hertzog
Pierre Habouzit

2e édition



EYROLLES

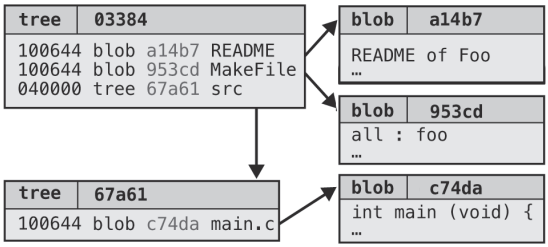
Git : définitions

Git permet d'enregistrer l'évolution d'une arborescence de fichiers (le ***répertoire de travail***) au sein d'un ***dépôt*** (le sous-répertoire `.git`). Développé en 2005 pour répondre aux besoins du développement du noyau Linux, il est puissant et polyvalent.

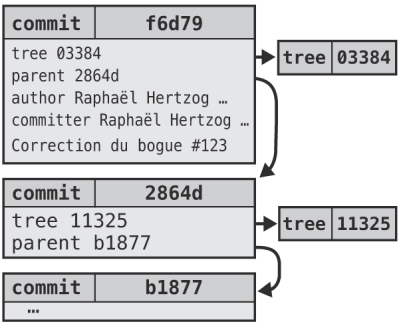
DÉFINITION **Système de gestion de versions distribué**
Avec les systèmes distribués, il n'est pas nécessaire d'avoir un dépôt central. Chaque intervenant dispose d'une copie complète du dépôt avec tout l'historique et peut effectuer autant de changements locaux qu'il le souhaite, sans devoir en référer au « serveur ». En pratique, on conserve des dépôts centraux pour faciliter les échanges et servir de référence commune. Mais les développeurs sont maîtres de ce qu'ils envoient sur ces dépôts, car le partage des changements est une opération totalement découplée de leur enregistrement.

DÉFINITION **Système de suivi de contenu**
Plus qu'un système de gestion de versions, Git est un système de gestion de contenus *(content tracker)*. Son fonctionnement interne reflète ce choix. Nous présentons rapidement les concepts associés, leur connaissance facilitant la compréhension de l'outil.

Un **dépôt Git** peut être vu comme une collection d'objets liés entre eux. Chaque objet est identifié par une chaîne de 40 caractères hexadécimaux, la somme de contrôle *(checksum)* SHA-1 de son contenu. Seuls les 5 premiers caractères figurent dans les exemples de ce mémento.
La représentation que Git se fait du répertoire de travail s'appuie sur deux types d'objets : des **données** *(blob)* et des **arborescences** *(tree)*. Une arborescence n'est rien d'autre que la représentation d'un répertoire, c'est-à-dire une liste de noms dont chacun est associé à des permissions et à un identifiant d'objet (ce dernier étant soit un fichier — objet de type « données », soit un répertoire — objet de type « arborescence »). Le répertoire de travail est donc lui-même un objet de type « arborescence ». S'il contenait les fichiers `README`, `Makefile` et `src/main.c`, il serait vu de la sorte :

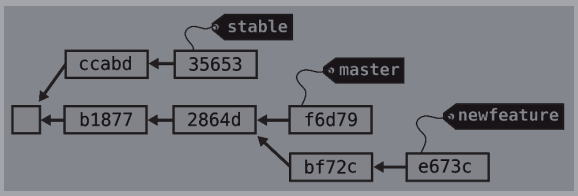


Un **historique de dépôt Git** n'est rien d'autre qu'une succession de versions du répertoire de travail (ce répertoire dans lequel vous travaillez et effectuez des modifications). Pour représenter cela, Git s'appuie sur un objet de type `commit` associant des informations telles que l'auteur, la date et le message descriptif, à une version du répertoire de travail, mais aussi et surtout à un commit « parent » représentant la version précédente (un commit fusionnant plusieurs branches aura donc plusieurs parents).



Ce document est la propriété exclusive de Aurore Mariscal (aurore.mariscal@hotmail.com) - jeudi 25 août 2022 à 10h33

En plus des **données**, **arborescences** et **commits**, Git manipule des **références**, qui sont autant de pointeurs vers des commits qui permettent de nommer des branches, des tags, etc. C'est ce qui permet à un dépôt Git de gérer en parallèle plusieurs **branches**, et donc plusieurs historiques. Une branche dans Git n'est pas un objet mais une référence qui pointe vers le dernier commit effectué dans la branche.



Mise en route

Première étape : indiquer à Git quelle identité sera enregistrée dans chaque commit (l'adresse par défaut *login@nom-machine* étant rarement la bonne).

```
git config --global user.email jean.dupont@example.com
git config --global user.name 'Jean Dupont'
```

L'option `--global` indique à Git d'enregistrer cette configuration dans `$HOME/.gitconfig` afin qu'elle soit partagée entre tous vos dépôts. Il sera toujours possible d'utiliser une identité différente au sein d'un dépôt particulier en utilisant ces commandes sans l'option **`--global`**.

Création d'un dépôt

N'importe quel répertoire peut devenir un dépôt Git : il suffit d'exécuter la commande **`git init`** dans le répertoire concerné. Ceci crée un répertoire **`.git`** avec les méta-informations nécessaires à Git. Peu importe que le répertoire de travail soit initialement vide ou non, le dépôt sera vierge et sans historique. L'inclusion de fichiers préexistants se fait séparément (cf. section « Modifications »). Un dépôt Git créé de la sorte combine dépôt et répertoire de travail. Pour créer un dépôt central dans lequel personne ne travaille mais utilisé par tous pour partager des modifications, il convient d'utiliser **`git init --bare`** ***`depot.git`***. Cette commande crée un répertoire **`depot.git`** ne contenant que les méta-informations Git, habituellement présentes dans le sous-répertoire `.git`.

REMARQUE L'extension `.git` du répertoire créé signale par convention un dépôt central sans répertoire de travail associé.

Téléchargement d'un dépôt existant

La plupart du temps, les dépôts Git sont préexistants et il convient de les télécharger pour pouvoir travailler dessus. La commande *ad hoc* est **`git clone`** ***`url`***. Le dépôt ainsi téléchargé est une copie complète, avec tout l'historique. L'URL à employer qui vous a été communiquée peut être de plusieurs formes :

`ssh://`*`login@git.example.com/chemin/depot`*`.git` *`login`*`@git.example.com:/chemin/depot.git`	Accès par SSH (méthode par défaut si le protocole est omis). Le préfixe login@ est optionnel.
`git://`*`git.example.com/chemin/depot`*`.git`	Accès par le serveur Git anonyme sur le port 9418 (git-server).
`http://`*`git.example.com/chemin/depot`*`.git`	Accès par HTTP.
`ftp://`*`git.example.com/chemin/depot`*`.git`	Accès par FTP.

REMARQUE On peut passer en dernier argument le nom du (nouveau) répertoire dans lequel le dépôt va être téléchargé, par défaut le dernier élément de l'URL, privé du suffixe `.git`.

Modifications (commandes de base)

Contrairement à d'autres systèmes de gestion de versions, lorsque vous exécutez `git commit`, Git n'enregistre pas les modifications depuis le répertoire de travail mais depuis « l'index », une zone tampon servant à préparer le prochain commit. Les commandes `git add` et `git rm` permettent d'intégrer certains changements du répertoire de travail dans l'index.

`git` **`add`** *`fichier`* `git` **`stage`** *`fichier`*	Ajoute un nouveau fichier à l'index, et/ou met à jour le contenu du fichier dans l'index.
REMARQUE `git stage` est synonyme de `git add` ; « stage » exprime mieux l'opération de mise à jour de l'index.	
`git add` **`-p`** *`fichier`* `git stage` **`-p`** *`fichier`*	Ajoute à l'index un sous-ensemble des changements présents dans le fichier. Les changements individuels sont extraits du patch global (cf. `git diff`) et sont proposés de manière interactive.
`git` **`rm`** *`fichier`*	Supprime un fichier du répertoire de travail ***et*** de l'index.
`git rm` **`--cached`** *`fichier`*	Supprime un fichier de l'index tout en le conservant dans le répertoire de travail.
`git rm` **`-r`** *`répertoire`*	Supprime du répertoire de travail *et* de l'index un répertoire et tous les fichiers qu'il contient, de manière récursive. `-r` fonctionne aussi avec `--cached`.
`git` **`mv`** *`ancien nouveau`* `mv` *`ancien nouveau`* `git add` *`nouveau`* `git rm` *`ancien`*	Renomme le fichier *`ancien`* en *`nouveau`*. Commande totalement équivalente à la séquence ci-contre (Git ne conserve pas les informations de renommage en interne).

`git` **`commit`**	Enregistre le contenu de l'index dans un nouveau commit.
`git commit` **`-a`**	Enregistre le contenu du répertoire de travail dans un nouveau commit. Cette commande appelle implicitement `git add` sur tous les fichiers modifiés (mais pas sur les nouveaux fichiers encore inconnus de Git !) et `git rm` sur tous les fichiers supprimés.
`git commit` ***`fichier`***	Crée un nouveau commit contenant uniquement les changements concernant le fichier indiqué. C'est le contenu du répertoire de travail qui est utilisé. L'index est mis à jour pour intégrer ces changements.
`git commit` **`--amend`**	Enregistre le contenu de l'index dans le dernier commit (`HEAD`). Cette opération remplace le dernier commit et ne doit pas être employée si ce commit a déjà été partagé avec d'autres utilisateurs. Elle permet simplement de corriger le dernier commit d'une branche privée.
`git` **`citool`**	Ouvre une application graphique (`git gui`) qui permet de préparer puis d'enregistrer un commit.
`git` **`status`**	Affiche l'état des fichiers du répertoire de travail et de l'index. Voir ci-dessous.

Avec Git on manipule couramment trois arborescences de fichiers : **`HEAD`** qui est la dernière version enregistrée de la branche courante, l'**index** et le **répertoire de travail** (*working directory* en anglais).
Pour suivre l'état des fichiers dans le répertoire de travail et dans l'index, Git fournit **`git status`**, qui classe les fichiers en différentes catégories :

- **non suivis** (fichiers non suivis, *untracked files*) pour les fichiers inconnus de Git, qui ne font partie ni du dépôt, ni de l'index (on peut les masquer en les listant dans `.gitignore`) ;
- **modifiés** (modifications qui ne seront pas validées, *changes not staged for commit*) pour les fichiers du dépôt Git qui présentent des modifications dans le répertoire de travail par rapport à l'index ;
- **enregistrés dans l'index** (modifications qui seront validées, *changes to be committed*) pour les fichiers du dépôt pour lesquels l'index contient des changements par rapport au dernier commit.

Un fichier peut être à la fois modifié et enregistré dans l'index, lorsque des changements interviennent après qu'on a appelé `git add` une première fois.

REMARQUE Pensez à **`git stash push`** et **`git stash pop`** pour conserver votre travail en cours sans le commiter, notamment si vous avez besoin de changer de branche.



Exploration (changements, historique)

Explorer des changements : `git diff`

git **diff**	Affiche la différence entre le contenu de l'index et celui du répertoire de travail. Ces changements ne seront pas commités par `git commit` sauf si on prend soin de les ajouter à l'index avec `git add` sur les fichiers listés.
`git` **diff --cached**	Affiche la différence entre le contenu du dernier commit et celui de l'index. Ces changements seront commités par `git commit`.
`git` **diff HEAD**	Affiche la différence entre le contenu du dernier commit et celui du répertoire de travail. Cela correspond à ce qui serait commité par `git commit -a`.
`git diff` *A B* `git diff` *A..B*	Affiche la différence entre le contenu pointé par *A* et celui pointé par *B*. *A* et *B* peuvent être des commits, tags, branches, objets de type « tree » ou « blob ».
`git diff` *A...B*	Affiche la différence entre le contenu de l'ancêtre commun à *A* et *B*, et celui de *B*.

REMARQUE Il est toujours possible de restreindre la génération des différences à un sous-ensemble des fichiers en les passant sur la ligne de commande après un double tiret, par exemple : **`git diff A B -- src/ Makefile`**

Explorer l'historique : `git log`, `git blame`, `gitk`

`git` **log [*A*]**	Affiche l'historique des changements (c'est à dire une liste de commits) accessibles depuis le commit pointé par *A*. Si *A* est omis, c'est `HEAD` qui est employé.
`git log [`*A*`]` **--*fichier***	Idem en restreignant l'affichage aux commits qui modifient le *fichier* indiqué. Très pratique pour voir les derniers changements ayant affecté un fichier !
`git` **log *A..B***	Affiche l'historique entres les deux commits pointés par *A* et *B*.
`git log` **--graph --oneline [*A*]**	Affiche un représentation textuelle de l'historique accessible à partir de *A* (`HEAD` si omis).
gitk `[`*A*`]`	Démarre un outil graphique de visualisation de l'historique accessible à partir de *A* (`HEAD` si omis).
`git` **blame** `[`*A*`] [--]` *fichier*	Affiche le contenu du *fichier* tel que présent dans l'arborescence de *A* (`HEAD` si omis) en indiquant pour chaque ligne le dernier commit à avoir modifié (ou introduit) celle-ci.

`git log` est une commande très puissante et dispose de plusieurs dizaines d'options. Les plus importantes sont reprises ci-contre. En outre, à la place de la référence optionnelle notée **`[A]`**, aussi bien `git log` que `gitk` acceptent toutes les options et syntaxes supportées par **`git rev-list`**, les plus importantes étant listées ci-dessous.

Options de **`git rev-list`** prises en charge par **`git log`** et **`gitk`**

`-n` *X*	Restreint l'affichage aux *X* derniers commits.
`--since=`*date* `--until=`*date*	Restreint à un sous-ensemble des commits en fonction de leur ancienneté.
`--author=`*modèle*	Restreint au sous-ensemble des commits dont l'auteur correspond au modèle indiqué.
`--grep=`*modèle*	Restreint au sous-ensemble des commits dont le message correspond au modèle indiqué.
`--all`	Étend l'ensemble des commits à afficher à tous ceux accessibles par une référence connue (branche locale, tag, branche distante).
`--branches[=`*modèle*`]`	Étend l'ensemble des commits à afficher à tous ceux accessibles par une branche locale dont le nom respecte le *modèle* indiqué (*toutes* les branches locales si *modèle* est omis).
`--remotes[=`*modèle*`]`	Similaire à **`--branches`** mais pour les branches distantes.
`--tags[=`*modèle*`]`	Similaire à **`--branches`** mais pour les tags.
`-S` *texte*	Limite la recherche à des commits qui introduisent/suppriment des lignes contenant ce texte.
intervalle	De nombreuses syntaxes permettent de désigner des ensembles de commits (voir la section « Aide syntaxique »).

Exploration (changements, historique)

Options de **`git log`**

`--decorate`	Affiche les noms des branches et tags à côté du commit correspondant.
`--oneline`	Affiche chaque commit sur une seule ligne (en utilisant l'identifiant de commit abrégé et la première ligne du message de log).
`-p`	Affiche le patch correspondant à chaque commit.
`--stat`	Affiche un `diffstat` correspondant aux changements du commit : la liste des fichiers modifiés avec des statistiques sur le nombre de lignes ajoutées et supprimées.

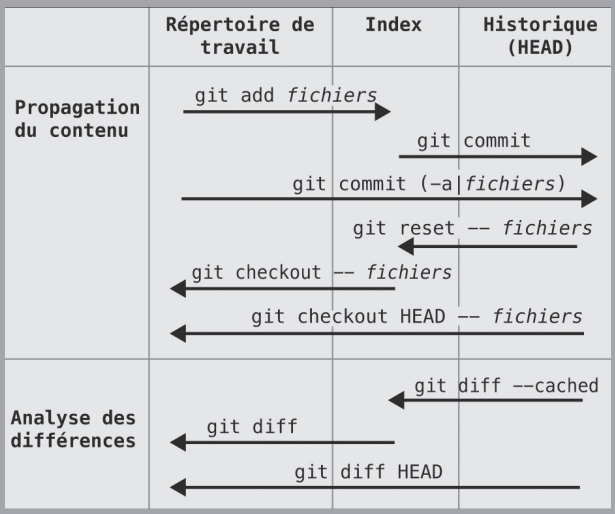
EXEMPLE **`git blame`**

```
$ git blame Makefile.am
[...]
841a630143 (Scott James Remnant 2005-03-11 09:00:14 +0000  23) EXTRA_DIST = \
cbf95b1350 (Guillem Jover       2009-10-25 23:06:46 +0100  24) .mailmap \
1a0b8ade8f (Guillem Jover       2009-03-02 06:56:25 +0200  25) ChangeLog.old \
fd3aae5215 (Raphaël Hertzog     2007-12-27 15:22:38 +0100  26) README.translators \
971e675a18 (Guillem Jover       2009-10-25 23:56:30 +0100  27) get-version \
c14a29ae99 (Guillem Jover       2009-09-23 03:21:16 +0200  28) doc/README.api \
aee5c77267 (Guillem Jover       2009-09-29 14:28:11 +0200  29) doc/coding-style.txt \
b3db5f47c0 (Sven Joachim        2010-07-25 23:34:03 +0200  30) doc/lcov-epilog \
b3db5f47c0 (Sven Joachim        2010-07-25 23:34:03 +0200  31) doc/lcov-prolog \
10440009b6 (Ian Jackson         2008-03-30 09:47:15 +0300  32) doc/triggers.txt \
[...]
```

EXEMPLE **`git log`**

```
$ git log --oneline --graph -decorate
* c8060af (HEAD -> master, origin/master) Drop non-required indexes
[...]
* 74bc441 Do not fail on broken Message-ID
*   f7b8537 Merge branch 'imagehover' into 'master'
|\
| * 0ad0143 Add title on images mouse hover
| * 8c9422e Resize ubuntu logo
* |   41cc146 Merge branch 'bug-787662' into 'master'
|\ \
| |/
|/|
| * a7c5cec Show BugsPanel for a package
|/
* b1ac881 settings: use lists instead of tuples for many settings
* dbe3ef2 utils: pass an explicit timezone
* dac264a settings: debug_toolbar needs a MIDDLEWARE setting too
*   bae2c34 Merge branch 'bts893670' into 'master'
|\
| * d178e14 UpdateAutoRemovalsStatsTask: import source package
|/
*   4d696bd Merge branch 'fix9' into 'master'
```

Schéma récapitulatif



Annulation, restauration et nettoyage

Effacer des changements non commités

`git` **`reset`** `git reset` **`--mixed`**	Efface les changements en attente de commit (autrement dit réinitialise l'index avec le contenu de `HEAD`). Les fichiers du répertoire de travail sont inchangés.
`git reset` **`--hard`**	Efface tous les changements présents dans l'index et dans le répertoire de travail (autrement dit, réinitialise l'index et le répertoire de travail avec le contenu de `HEAD`). **Attention, cette commande est destructrice !** Il est impossible de récupérer les changements qui étaient présents dans le répertoire de travail.
`git reset [--] fichier...`	Efface uniquement les changements en attente de commit qui affectaient le *`fichier`* (autrement dit restaure l'état du *`fichier`* dans l'index, à son état dans `HEAD`). Cette commande est donc la réciproque de `git add`.
`git` **`checkout`** `[--] fichier...`	Efface les changements du *`fichier`* du répertoire de travail par rapport à sa copie dans l'index (autrement dit restaure le *`fichier`* du répertoire de travail à son état dans l'index).
`git checkout HEAD [--] fichier...`	Efface les changements du *`fichier`* du répertoire de travail *et* de l'index (autrement dit restaure le fichier du répertoire de travail et de l'index à son état dans `HEAD`). Équivalent de **`git reset -- fichier && git checkout -- fichier`**.

REMARQUE Pour ces trois dernières commandes, l'option **`-p`** permet de choisir interactivement le sous-ensemble des changements à effacer.

Restaurer l'état d'un fichier donné

`git reset A [--] fichier...`	Restaure l'état du *`fichier`* dans l'index, à son état dans le commit *A*.
`git checkout A [--] fichier...`	Restaure le *`fichier`* du répertoire de travail (et de l'index) à son état dans le commit *A*.

REMARQUE Pour ces deux commandes, l'option **`-p`** permet de choisir interactivement le sous-ensemble des changements à restaurer.

Revenir sur des changements commités

Lorsque **`git reset`** prend en paramètre un commit (et aucun fichier), il fait pointer `HEAD` sur le commit indiqué. Selon les options, il modifie ou non l'index et le répertoire de travail. Cette commande permet donc de revenir en arrière dans l'historique en laissant ou non la possibilité de recréer les derniers commits à partir des changements encore présents dans l'index et/ou le répertoire de travail.

Commande	Commit	HEAD	Index	Répertoire de travail
`git reset --soft A`	A	A	inchangé	inchangé
`git reset --mixed A` `git reset A`	A	A	A	inchangé
`git reset --hard A`	A	A	A	A

Nettoyage

`git clean` sert à supprimer du répertoire de travail les fichiers non suivis (et non ignorés) par Git. Il faut systématiquement passer l'option **`-f`** pour réaliser cette opération destructrice (sauf si l'option de configuration **`clean.requireForce`** est à **`false`**). On peut aussi employer l'option **`-n`** pour simplement afficher ce qui aurait été supprimé (si l'on avait mis **`-f`** au lieu de **`-n`**).
L'option **`-x`** inclut également les fichiers ignorés (listés dans `.gitignore`) et l'option **`-d`** inclut les répertoires en plus des fichiers.



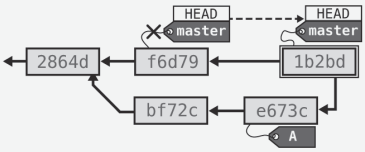
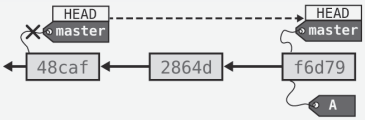



Gestion des branches

Espace de nommage

Pour Git, une ***branche*** n'est qu'un nom pointant sur un commit. Ce pointeur évolue au fur et à mesure que la branche évolue, contrairement au *tag* qui a vocation à être un « pointeur statique ».
Les branches (locales et distantes) et les tags partagent un espace de nommage hiérarchique commun, le sous-répertoire **`.git/refs`** se subdivisant entre autres en ***heads*** (pour les branches locales), ***remotes*** (pour les branches distantes) et ***tags*** (pour les tags).
Ainsi, le nom complet de la branche locale *master* est en réalité `refs/heads/master`, mais Git accepte que l'on omette les deux premiers niveaux de la hiérarchie et recherche alors automatiquement la référence dans les 3 hiérarchies standards (et se plaint s'il y a une ambiguïté).

Commandes

`git` **`branch`**	Liste toutes les branches locales. Avec **`-r`**, liste toutes les branches distantes. Avec **`-a`**, liste toutes les branches, locales *et* distantes.
`git branch` *`branche`* `[`*`A`*`]`	Crée une nouvelle *`branche`* pointant sur *A*, ou sur `HEAD` si *A* est omis. Attention, cette commande ne modifie pas la branche courante.
`git` **`checkout`** *`A`*	Fait de *A* la branche courante et met à jour le répertoire de travail en conséquence. Cette opération peut échouer si des changements non commités ne peuvent être conservés au sein de A. Si A n'existe que comme une branche distante, cela va créer une branche locale du même nom (pointant sur le même commit) avant d'en faire la branche courante.
`git` **`checkout -b`** *`branche`* `[`*`A`*`]`	Crée une nouvelle branche *`branche`* pointant sur *A*, ou sur `HEAD` si *A* est omis, puis fait de *`branche`* la branche courante et met à jour le répertoire de travail en conséquence.
`git` **`merge`** *`A`*	Fusionne *A* dans la branche courante. Habituellement cette commande crée un nouveau commit représentant le résultat de cette fusion, celui-ci disposant alors de 2 commits parents.  Si *A* contient déjà tous les changements de la branche courante, la fusion se fait par *fast forward* : la branche courante est simplement mise à jour pour pointer vers *A*, aucun nouveau commit n'est créé. 
`git merge` **`-s ours`** *`A`*	Fusionne *A* dans la branche courante, mais en ignorant tous les changements qu'elle introduit, juste pour créer le point de parenté. **REMARQUE** Utile pour fusionner une branche contenant des changements inutiles (par exemple, un correctif déjà présent sous une autre forme) qui, autrement, créeraient de nombreux conflits.
`git` **`mergetool`**	Outil facilitant la résolution des conflits qui peuvent survenir lors de la fusion de branches.
`git branch` **`-d`** *`branche`*	Supprime la branche *`branche`*. Cette commande échoue si *`branche`* n'est pas accessible dans l'historique de la branche courante. On peut toutefois forcer cette suppression en employant **`-D`** au lieu de **`-d`**.
`git branch` **`-m`** `[`*`ancien`*`]` *`nouveau`*	Renomme la branche *`ancien`* (ou la branche courante si *`ancien`* est omis) en *`nouveau`*. Cette commande échoue si *`nouveau`* existe déjà. On peut toutefois écraser *`nouveau`* en utilisant **`-M`** au lieu de **`-m`**.

Résolution de conflits

Lorsque les deux branches fusionnées modifient une même partie d'un fichier, cela crée un **conflit** qu'il convient de résoudre manuellement. Git interrompt alors le processus sans créer le commit de fusion. Les fichiers contenant les modifications conflictuelles sont clairement identifiés comme tels dans la sortie de **`git status`** (marqués modifiés des deux côtés ou *both modified* dans la section Chemins non fusionnés ou *Unmerged paths*).
Chaque fichier contenant des conflits doit être édité pour définir le contenu souhaité. Les lignes conflictuelles se trouvent entre les marqueurs `<<<<<<<`, `=======` et `>>>>>>>`. Une fois ces marqueurs enlevés et le bon contenu mis en place, il faut enregistrer le résultat de la fusion dans l'index avec `git add` *`fichier`*. Un `git commit` permet alors de terminer le processus de fusion. Si vous souhaitez abandonner une fusion trop difficile, **`git merge --abort`** restaurera le répertoire de travail dans son état initial. Il est possible d'utiliser `git mergetool` pour gérer cette résolution de conflits de façon semi-automatique. La suppression d'un fichier peut également engendrer un conflit lorsqu'on essaie de fusionner avec une branche qui modifie ce même fichier. Il faut alors soit confirmer la suppression avec **`git rm`** soit restaurer le fichier avec **`git add`**.

Partage et publication

Gestion des dépôts distants

À partir du moment où plusieurs personnes collaborent sur un même projet, on a inévitablement plusieurs dépôts Git dont certains sont publics pour faciliter le partage du travail réalisé. Git peut interagir avec ces dépôts distants en intégrant une copie de leurs branches dans son espace de nommage — chaque dépôt s'octroyant la hiérarchie `refs/remotes/`*`nom-dépôt-distant`*. Les commandes ci-dessous permettent de configurer ces dépôts distants et d'interagir avec eux.

REMARQUE Lorsque le dépôt Git est téléchargé par **`git clone`**, un dépôt distant **`origin`** est automatiquement mis en place. La configuration par défaut de ce dépôt distant récupère toutes les branches distantes dans des copies locales accessibles *via* `refs/remotes/origin/`*`nom-branche`*.

`git` **`remote`**	Liste les dépôts distants configurés.
`git remote` **`add`** *`dépôt url`*	Ajoute un nouveau *`dépôt`* distant accessible à l'*`URL`* indiquée (cf. section « Mise en route » pour les URL*s* valides).
`git remote` **`update`**	Met à jour la copie locale de tous les dépôts distants en exécutant `git fetch` sur chacun des dépôts configurés. Avec l'option **`--prune`**, les branches distantes ayant disparu du dépôt distant sont également supprimées.
`git` **`fetch`** *`dépôt`*	Met à jour la copie locale du *`dépôt`* distant.
`git` **`push`** [*`dépôt`*]	Envoie les changements de la branche courante vers la branche de même nom du *`dépôt`* distant (s'il est omis, c'est le dépôt distant associé à la branche courante qui est utilisé, sinon c'est `origin` qui est employé). Ce comportement par défaut peut être personnalisé pour chaque dépôt distant avec le paramètre `remote.`*`dépôt`*`.push`. En l'absence de ce paramètre, Git utilise le paramètre `push.default` pour connaître le comportement à adopter. Voir la section « Configuration » pour plus d'informations.
`git push` *`dépôt L`*`:`*`D`*	Crée ou met à jour la branche distante *`D`* avec les changements contenus dans la branche locale *`L`*.
`git push` *`dépôt L`*	Si « `:`*`D`* » est omis, alors la branche distante a le même nom que la branche locale (*`L`*).
`git push` *`dépôt :D`*	Si *`L`* est omis, la branche distante *`D`* est supprimée. **REMARQUE** On peut employer `HEAD` à la place de *`L`* pour indiquer le nom de la branche courante sans devoir le saisir.
`git` **`pull`**	Met à jour la branche distante que la branche courante suit, puis intègre les nouveaux changements dans la branche courante. Avec l'option **`--rebase`**, la mise à jour de la branche locale est effectuée avec `git rebase` *`branche-distante`* au lieu de `git merge` *`branche-distante`*.



Partage de changements

Les changements sont en général destinés à être partagés, qu'on veuille les soumettre pour inclusion au mainteneur du projet ou qu'on souhaite des retours sur le travail effectué. Il n'y a que deux manières de procéder. Soit on publie une branche sur un dépôt Git (accessible par les personnes avec lesquelles on souhaite partager nos changements), soit on met à disposition des **patches** correspondant aux changements à partager.

`git` **`format-patch`** *A*	Extrait tous les commits de la branche courante qui ne sont pas dans la branche *A* sous forme de patchs stockés dans le répertoire courant (ou dans le répertoire précisé avec l'option **`-o`** ou **`--output-directory`**).
`git` **`send-email`** *fichier* `git` **`send-email`** *répertoire* `git` **`send-email`** *branche*	Envoie tous les *fichiers* patch (ou tous les fichiers patch contenu dans le *répertoire*) par courrier électronique. La troisième forme acceptant une *branche* utilise directement **`git-format-patch`** *branche* pour générer les patchs à envoyer. De nombreuses options existent pour définir les différents champs du courrier expédié. Citons l'option **`--to`** qui indique le destinataire principal.
`git` **`push`**	(Commande déjà traitée dans la section « Gestion des dépôts distants »). Moyen le plus fréquent de partager une branche en l'envoyant sur un dépôt public.

Publication

`git` **`tag`** *nom*	Crée un tag *nom* et l'associe au commit courant. Un tag permet de retrouver facilement l'état du projet à un moment clef (lorsqu'une nouvelle version est publiée notamment). L'option **`-s`** permet de créer un tag signé avec GnuPG. L'option **`-d`** supprime le tag nommé.
`git` **`archive`** `[--prefix=`*valeur*`]` `-o` *fichier*`.tar.gz` *A*	Crée une archive *fichier*`.tar.gz` correspondant à une copie du contenu du dépôt tel qu'il est dans le commit pointé par *A*. Outre les archives `.tar.gz`, il est possible de générer des archives `.zip`. L'option **`--prefix`** définit un préfixe à ajouter à tous les noms de fichiers. En choisissant « projet/ » comme préfixe, on place tous les fichiers dans un répertoire « projet » au sein de l'archive générée.

Modifications (commandes avancées)

`git` **`apply`** *patch*	Applique le *patch* (cf. plus haut `format-patch`) au répertoire de travail. Si `--index` est passé, applique également *patch* à l'index. Si `--cached` est passé, alors le *patch* n'est appliqué qu'à l'index et pas au répertoire de travail. Cette commande ne crée pas de commit.

REMARQUE Si une partie du patch ne s'applique pas proprement, alors l'arborescence reste inchangée (contrairement à la commande `patch` qui applique tout ce qu'elle peut et laisse des fichiers de rejet derrière elle, un comportement qui peut toutefois être obtenu *via* l'option `--reject`).



Modifications (commandes avancées)

`git` **`am`** *`boîte-aux-lettres`*
`git` **`am`** *`fichiers-patch`*

Applique une série de patchs reçus par courriel et crée un commit pour chaque patch. On peut lui passer une *`boîte-aux-lettres`* au format Mailbox ou Maildir, ou bien directement un ensemble de patchs générés avec `git format-patch`. Les informations du commit sont directement extraites du message électronique (remarquez que la sortie de `git format-patch` génère effectivement un courriel).

REMARQUE Si le processus s'interrompt à cause d'un patch qui ne s'applique pas, vous pouvez appliquer ce dernier à la main et l'enregistrer dans l'index avant de relancer le processus avec **`git am --continue`**. Pour ignorer le patch problématique, on fera **`git am --skip`**. Pour abandonner l'opération entière : **`git am --abort`**.

`git` **`cherry-pick`** *A…*

Pour chaque commit indiqué (*A*…), crée un nouveau commit dans la branche courante et duplique le contenu du commit indiqué. Alors qu'une fusion intègre tous les changements d'une branche, cette commande permet de choisir exactement quels changements intégrer dans la branche courante.

REMARQUE L'opération peut générer des conflits qu'il convient de régler en enregistrant le résultat désiré dans l'index, puis en appelant **`git cherry-pick --continue`**. Si on veut annuler l'opération, effectuer **`git cherry-pick --abort`**.

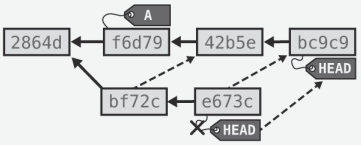
`git` **`revert`** *A…*

Pour chaque commit indiqué (*A*…), crée un nouveau commit dans la branche courante qui annule les changements apportés par le commit indiqué.

REMARQUE `git revert` est l'opération inverse de `git cherry-pick`. Il est implémenté avec la même fonction que `git cherry-pick`, mais c'est le patch inversé du commit indiqué qui est employé (un ajout devient une suppression et vice-versa).

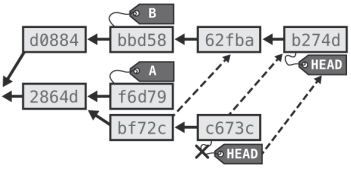
`git` **`rebase`** *A*

Réécrit la branche courante de telle manière que tous ses commits soient des descendants de *A*. Pour cela, tous les commits de la branche courante (depuis le premier descendant du premier ancêtre commun à *A* jusqu'à `HEAD`) sont recréés (comme avec `git cherry-pick`). Le plus ancien commit devient le descendant direct de *A*.



`git` **`rebase --onto`** ***B*** *A*

Réécrit la branche courante (dérivée de *A*) de telle manière que tous ses commits soient des descendants de *B*. Pour cela, tous les commits de la branche courante (depuis le premier descendant du premier ancêtre commun à *A* jusqu'à `HEAD`) sont recréés (comme avec `git cherry-pick`). Le plus ancien commit devient le descendant direct de *B*.



`git` **`rebase -i`** *A*

L'option **`-i`** permet de modifier de manière interactive l'historique des changements en réordonnant les commits, en modifiant leur message (**`reword`**), en fusionnant certains commits dans d'autres (**`squash`**, **`fixup`**), en modifiant ou découpant des commits (**`edit`**), voire en en supprimant certains. `git commit --fixup=`*`B`* (ou `--squash=`*`B`*) crée un commit marqué comme étant à fusionner dans *B* au cours du prochain `git rebase --autosquash -i` qui incluera ce commit.

REMARQUE La différence entre `squash` et `fixup` est que le premier ouvre un éditeur pour modifier le message de commit résultant de la fusion, alors que le second conserve le message du commit ciblé.

Aide syntaxique : treeish et intervalles

Quand nous mentionnons *A* (ou *B*) dans une description de commande, il est possible d'utiliser une multitude de syntaxes pour désigner un commit. Si on se contente souvent de donner le nom d'une branche pointant implicitement sur le dernier commit qu'elle contient, il y a bien des situations où l'on voudra désigner d'autres commits. La documentation (anglaise) de Git désigne ces paramètres (*A* ou *B*) comme étant des ***treeish***, ce que l'on pourrait traduire par « qui ressemble à une arborescence ». Cette formulation signifie que Git accepte toute expression lui permettant d'identifier un objet de type arborescence (« tree »). Voici les syntaxes les plus utiles :

chaîne-hex	Toute chaîne de caractère hexadécimale (`0-9`, `a-f`) sera interprétée comme étant un identifiant de commit. Si la chaîne est plus courte que les 40 caractères requis par une somme de contrôle SHA1, elle est interprétée comme étant le début d'un tel identifiant (en pratique, l'identification ne nécessite souvent que 5 à 7 caractères). **EXEMPLE** `dae86e1950b1277e545cee180551750029cfe735, dae86e`
`refs/heads/`*foo* `refs/tags/`*bar* `refs/remotes/`*depot*`/`*baz*	Nom complet d'une référence symbolique désignant respectivement la branche locale *foo*, le tag *bar*, la branche distante *baz* dans le dépôt distant *depot*.
foo	Nom court d'une référence symbolique, que Git va résoudre en prenant le premier objet valide dans la liste suivante : **`$GIT_DIR/`***foo* (gère le cas des références `HEAD` et `*_HEAD`) **`refs/`***foo* **`refs/tags/`***foo* **`refs/heads/`***foo* **`refs/remotes/`***foo* **`refs/remotes/`***foo*`/HEAD` **EXEMPLE** `master`, `heads/master`, `refs/heads/master`, `origin/master`, `remotes/origin/master`, `refs/remotes/origin/master`
A`^`	Le premier commit parent du commit pointé par *A*. On peut ajouter plusieurs « ^ » pour traverser l'historique sur plusieurs niveaux, mais la syntaxe A~n généralise cette opération sur *n* niveaux.
A`^`***n***	Le *n*-ième parent du commit pointé par *A*. Pour des valeurs $n > 1$, cette syntaxe n'est valable que pour des commits fusionnant plusieurs branches.
A`~`***n***	Le *n*-ième commit en arrière dans l'historique. `A~1` désigne le même commit que `A^`, de même pour `A~2` et `A^^`, etc.
foo`@{`*n*`}`	Le commit sur lequel pointait *foo* en revenant *n* « changements » de *foo* en arrière (par « changement », on entend toute opération qui a fait pointer *foo* sur un commit différent). Si *foo* est omis, on utilise le nom de la branche courante. Ex. : `master@{2}`, `@{3}` **REMARQUE** Cette syntaxe utilise le *reflog* de git, une sauvegarde automatique (sur une durée configurable) des anciennes versions des branches présentes dans le dépôt.
foo`@{`*date*`}`	Le commit sur lequel pointait *foo* à la *date* (et heure) indiquée. La date peut s'exprimer de nombreuses manières différentes comme *yesterday*, *3 days 1 hour ago* ou simplement *2012-02-09 15:32*. Ex : `master@{1 week ago}`, `HEAD@{14:30}` **REMARQUE** Cette syntaxe utilise également le *reflog* de git, ce qui veut dire que cela renvoie l'état de la branche *locale* à la date indiquée, et non pas l'état de l'éventuelle branche distante associée.
A`:`*fichier*	Le *fichier* de l'arborescence associée au commit pointé par *A*. Contrairement aux autres syntaxes présentées, cette syntaxe ne renvoie pas un objet de type commit mais un objet de type *blob*. S'utilise notamment avec `git show` qui sait afficher le contenu de n'importe quel objet, ou encore avec `git diff` qui sait comparer deux fichiers arbitraires.

REMARQUE Retrouvez toutes ces syntaxes dans la page de manuel `gitrevisions(7)`.



Syntaxe pour décrire des ensembles de commits sous forme d'intervalles (*ranges*)

A	*A* et tous ses ancêtres. **REMARQUE** Lorsque Git attend un intervalle, désigner un commit inclut automatiquement tous les commits ancêtres.
A ^B *B..A*	*A* et tous ses ancêtres, sauf *B* et ses ancêtres.
A...B	*A* et *B* et tous leurs ancêtres sauf les ancêtres communs aux deux.
A^!	Le commit *A* tout seul.

L'arme ultime pour isoler le commit fautif : git bisect

Un bogue qu'on peut reproduire de manière fiable est un bogue à moitié corrigé, dit-on souvent. C'est d'autant plus vrai si ce bogue est une régression, car Git fournit une commande formidable pour identifier le commit fautif, **`git bisect`**, qui implémente une recherche par dichotomie. Il faut donc spécifier un ensemble de commits à tester en indiquant une version connue comme étant affectée par le problème (`bad`) et une version non affectée (`good`). Git extrait le commit médian de cet ensemble et attend votre verdict : cette version est-elle affectée ou non ? Partant de là, Git réduit de moitié l'espace des commits à tester. Après quelques itérations, Git indique le premier commit affecté par le problème.

`git bisect start`
`git bisect start [`*`A`*
`[`*`B`*`...]]`
	Démarre une recherche par dichotomie. Si *A* est précisé, il s'agit d'un « mauvais » commit (c'est-à-dire affecté par le problème). Si *B* est précisé, il s'agit d'un « bon » commit (non affecté par le problème).
	Pour restreindre l'ensemble des commits à ceux qui modifient des fichiers, il est possible d'en indiquer les chemins (fichiers ou répertoires) après deux-tirets (`--`).

`git bisect bad [`*`A`*`]`	Enregistre *A* comme étant affecté par le problème, il s'agit donc d'une *borne haute* à l'ensemble de commits à tester. Si *A* est omis, c'est `HEAD` qui est utilisé.
`git bisect good [`*`A`*`]`	Enregistre *A* comme étant non affecté par le problème, il s'agit donc d'une *borne basse* à l'ensemble de commits à tester. Si *A* est omis, c'est `HEAD` qui est utilisé.
`git bisect skip [`*`A`*`]`	Enregistre *A* comme étant non testable. Si *A* est omis, c'est `HEAD` qui est utilisé. **ATTENTION** Si le commit fautif est adjacent à un commit non testable, cela peut empêcher Git de l'identifier.
`git bisect reset`	Termine ou interrompt la recherche par dichotomie. Le répertoire de travail et le dépôt sont remis dans leur état initial.
`git bisect run` *`commande`*	Automatise la recherche à l'aide d'une *`commande`* externe. Pour chaque test, Git exécute *`commande`* et utilise le code de retour pour déterminer le statut du commit : **0**, le commit est « bon » ; **1-127** (*sauf 125*), le commit est « mauvais » ; **125**, le commit est non testable ; **128-255**, une erreur s'est produite et le processus est interrompu.
`git bisect visualize`	Affiche l'ensemble des commits restant à tester dans **`gitk`**.



L'arme ultime pour isoler le commit fautif : git bisect

`git bisect log`	Affiche un historique de la recherche par dichotomie, qui peut être sauvegardé pour être rejoué (voir `git bisect replay`). En cas d'erreur sur l'une des réponses, on peut modifier l'historique avant de le rejouer.
`git bisect replay` *fichier*	Extrait la liste des bons et mauvais commits depuis le *fichier* et continue la recherche du commit fautif à partir de là.

EXEMPLE **git bisect pour identifier le commit fautif**

```
$ git bisect start -- src/archives.c
$ git bisect bad 1.16.0
$ git bisect good 1.15.7
Bissection : 35 révisions à tester après ceci (à peu près
5 étapes)
[5d74139fda1e4850c436765131b949232b54cc80] Use fdio in-
stead of ad-hoc code to handle interrupted I/O
$ git bisect good
Bissection : 17 révisions à tester après ceci (à peu près
4 étapes)
[ccb9d03ffd24c0e7319872bf190ce99dbb37b18e] libdpkg: Rename
TarInfo to tar_entry
[…]
$ git bisect bad
d25407536dbed4cad2943187b36fbb6c92a6b5ab is the first bad
commit
[…]
$ git bisect reset
[…]
```

Utiliser Git avec un dépôt Subversion : git svn

`git svn` crée un dépôt Git local s'interfaçant avec un dépôt distant Subversion. Git est utilisé normalement pour travailler et enregistrer des changements, quelques commandes spécifiques permettant de récupérer et d'envoyer les changements depuis/vers le serveur Subversion.

ATTENTION N'utilisez pas les fonctionnalités de fusion de Git car elles ne peuvent se transposer aisément à Subversion. **`git svn`** s'appuie à la place sur la fonctionnalité **`rebase`** pour maintenir une branche à jour.

`git svn clone` *svn-url* `git svn clone -s` *svn-url*	Crée un dépôt Git miroir du répertoire pointé par l'URL ***`svn-url`***. Avec l'option **`-s`**, les branches et les tags SVN sont également importés. Dans ce cas, l'URL indiquée est implicitement suffixée de `trunk`, `branches` et `tags`. Lorsque cette structure n'est pas respectée, les options **`-T`**, **`-b`** et **`-t`** permettent d'indiquer l'URL pour chacun de ces 3 répertoires standard.
`git svn rebase`	Met à jour la copie locale du dépôt Subversion puis `rebase` la branche locale courante sur la branche SVN mise à jour. C'est l'équivalent de **`svn update`**.
`git svn fetch`	Met à jour la copie locale du dépôt Subversion sans toucher à la branche locale courante.
`git svn dcommit`	Envoie les changements locaux (de la branche courante) vers le dépôt Subversion. C'est l'équivalent de plusieurs **`svn commit`** successifs sur tous les changements non encore publiés.
`git svn branch` *foo*	Crée une nouvelle branche Subversion nommée *foo* qui démarre du commit actuel.
`git svn tag` *foo*	Crée un nouveau tag Subversion nommé *foo* qui pointe sur le commit actuel.



Fichiers de configuration

Les fichiers de configuration de Git ont un format proche du `.ini` classique. On les modifie avec **`git config`** ou dans votre éditeur favori.

`/etc/gitconfig`	Ficher de configuration système, normalement paramétré par le packager ou par l'administrateur système. Modifié par `git config` **`--system`** *`option valeur`*
`~/.gitconfig`	Ficher de configuration global de l'utilisateur. Modifié par `git config` **`--global`** *`option valeur`*
`depot/.git/config`	Fichier de configuration spécifique au dépôt local. Modifié par `git config` *`option valeur`* depuis n'importe quel répertoire du dépôt local.

En cas de définition multiple d'un paramètre de configuration : le fichier du dépôt local est prioritaire sur le fichier de l'utilisateur, lui-même prioritaire sur le fichier de configuration système. L'intégralité des paramètres de configuration de Git est documentée dans la page de manuel `git-config(1)`.

Principales options de configuration de Git

`alias.`*`nom`*	Définit *nom* comme une nouvelle commande Git qui invoque en réalité la commande indiquée par la valeur de cette option (voir la section suivante pour des alias très utiles).
`user.email`	Adresse électronique utilisée pour les commits.
`user.name`	Nom complet utilisé pour les commits.
`user.signingkey`	Identifiant de la clef GPG à utiliser pour signer les tags, les commits ou les pushs. En général correctement déduit de `user.email`.
`core.editor`	Permet de choisir l'éditeur utilisé pour les commits. Attention, cet éditeur ne doit pas passer en arrière-plan une fois invoqué.
`merge.conflictstyle`	Peut être positionné sur **`diff3`**, ce qui permet d'avoir en plus des marques de conflit avec la version « de gauche » (marqueurs `<<<<<`) et « de droite » (marqueurs `>>>>>`), la version de l'ancêtre commun (entre marqueurs `|||||` et `=====`).
`color.ui`	Contrôle l'usage des couleurs dans la sortie des différentes commandes qui savent mettre en valeur ce qu'elles affichent. La valeur recommandée est **`auto`** qui active la couleur lorsque la commande est exécutée sur un terminal mais qui la désactive dans le cas où la sortie est redirigée vers un autre programme (dans un *pipe*). **`never`** prescrit de ne jamais utiliser la couleur, **`always`** de toujours le faire.
`format.pretty`	Format de présentation des commits employé par les commandes `git log`, `git show` et `git what-changed`. La valeur par défaut est **`medium`**, les autres valeurs possibles étant **`oneline`**, **`short`**, **`full`**, **`fuller`**, **`email`**, **`raw`**.
`merge.tool`	L'outil invoqué par `git mergetool` pour résoudre les conflits. `git mergetool --help` liste les outils pris en charge.
`pretty.`*`nom`*	Permet de créer des formats personnalisés pour `git log` sous le nom *nom*, qu'on utilise ensuite *via* `git log --pretty=`*`nom`*.
`remote.`*`dépôt`*`.push`	Définit explicitement la liste des références envoyées par défaut au dépôt distant lorsque l'on fait `git push` *`dépôt`*.





Principales options de configuration de Git

`push.default`	Définit le mode de fonctionnement de `git push [`*`dépôt-distant`*`]` sans arguments (`origin` est utilisé comme *`dépôt-distant`* si ce paramètre est omis) lorsque `remote.`*`dépôt-distant`*`.push` n'est pas défini. Les valeurs possibles de `push-default` sont : ■ **`nothing`** : ne fait rien ; ■ **`current`** : pousse la branche courante vers une branche de même nom dans le dépôt distant ; ■ **`upstream`** : pousse la branche courante vers la branche du dépôt distant qu'elle suit ; ■ **`simple`** : lorsqu'on pousse vers le dépôt suivi, fonctionne comme upstream mais refuse de pousser dans une branche de nom différent. Si non, fonctionne comme current lorsqu'on pousse dans un autre dépôt. C'est le comportement par défaut, le plus sûr pour les débutants. ■ **`matching`** : pousse les branches qui correspondent entre le dépôt local et le dépôt distant (comportement par défaut).
`rebase.autosquash`	Active par défaut l'option `--autosquash` de la commande `git rebase -i`.
`status.showUntrackedFiles`	Si cette option est à `no`, `git status` n'affiche plus les fichiers inconnus de Git (non gérés par celui-ci). Option intéressante pour les très gros dépôts où la commande `git status` est assez longue à s'exécuter.
`url.`*`base`*`.` `pushInsteadOf`	Par défaut Git utilise la même URL pour télécharger un dépôt que pour y envoyer des changements. Lorsque ce n'est pas possible, et pour éviter de devoir configurer une URL spécifique (dans `remote.`*`name`*`.pushurl`) pour chaque dépôt, cette option indique à Git de réécrire l'URL utilisée pour l'envoi en remplaçant le préfixe indiqué dans la valeur associée par *`base`*. **EXEMPLE** `git config` **`--global`** `url.git@salsa.debian.org:.`**`pushInsteadOf`** `https://salsa.debian.org/`

Alias fréquemment employés

Pour créer un alias, il faut exécuter :
`git config --global alias.`*`nom commande`*.

Alias	Commande	Explications
`ci`	`commit`	Version courte de cette commande fréquemment utilisée.
`co`	`checkout`	Idem.
`st`	`status`	Idem.
`br`	`branch`	Idem.
`tip`	`log -n 1 --abbrev-commit --decorate`	Affiche le dernier commit de la branche courante (ou de la référence indiquée si l'on exécute `git` **`tip`** *A*).
`unstage`	`reset HEAD --`	Supprime les changements en attente dans l'index. Si l'on indique une liste de fichiers, seuls ceux-ci sont affectés.
`staged`	`diff --cached`	Affiche les changements en attente dans l'index.
`fix`	`commit --amend`	Corrige le dernier commit.



Bonnes pratiques

Cohérence et taille des commits : enregistrez des changements cohérents et minimaux
Un commit doit contenir des **changements cohérents, qui vont ensemble**. Ainsi les corrections de deux bogues différents doivent en règle générale se retrouver dans deux commits différents. De plus, les petits commits sont bien plus **faciles à relire** pour les autres développeurs. En cas de problème, cela permet aussi de retrouver plus facilement le changement fautif (grâce à `git bisect` notamment).
Si des changements hétérogènes sont mélangés dans votre répertoire de travail, enregistrez-les par petits bouts :
`git add -p`, `git checkout -p`, `git rebase -i`, `git reset -p`.

Rythme des commits : faites des commits fréquents et prévenez les conflits
Cela permet d'enregistrer des changements cohérents et de petite taille, et de partager votre code plus souvent avec les autres développeurs.
En intégrant vos développements en cours plus régulièrement, vous prévenez les conflits importants qui arrivent souvent en cas d'intégration tardive d'une branche. Avoir peu de commits contenant beaucoup de modifications, et ne les partager que rarement, génère des conflits plus gros et plus difficiles à résoudre.
Enfin, avant de partager votre code, utilisez si nécessaire `git rebase -i` pour réordonner, reformuler et agréger des commits afin de présenter un historique lisible aux autres programmeurs, dans lequel chaque commit représente un ensemble minimal et cohérent de modifications clairement documentées.

N'enregistrez pas du travail à moitié fini : utilisez `git stash push` **et** `git stash pop`
Un commit doit contenir un travail « complet ». Ceci n'est pas en opposition avec le fait qu'il faut faire des commits de petite taille : il ne s'agit pas de faire en un seul commit une fonctionnalité compliquée, mais de découper la fonctionnalité en plusieurs petites tâches, et d'enregistrer un changement par petite tâche.
Ne faites pas de commit le soir avant de quitter le bureau juste pour avoir un répertoire de travail propre. Dans ce genre de cas, recourez à `git stash`, car les commits temporaires ont la fâcheuse tendance à se retrouver dans les dépôts centraux à cause de `git push` trop hâtifs par exemple…

Documentation : écrivez de bons messages de commit
La gestion de versions perd son sens si les messages de commit sont de mauvaise qualité. Les messages s'adressent tant aux autres qu'à vous-même, ils seront lus par toutes les personnes qui vont ausculter l'historique et se demander invariablement « Pourquoi ce changement ? Qu'est-ce qui a changé ? » plutôt que « Comment fonctionne le code ? » .
Un bon message de commit commence par un résumé d'une cinquantaine de caractères qui décrit le changement. Suit une ligne vide qui le sépare du message détaillé, qui sera aussi long que nécessaire. Un bon message précise donc : 1) ce qui a **motivé** le changement (bogue, nouvelle fonctionnalité, nettoyage, optimisation…) ; 2) ce qui a **changé** par rapport à la version précédente.
Ce n'est pas le lieu pour décrire l'implémentation en détail, elle doit l'être dans le code sous forme de commentaires.

N'enregistrez jamais de changements non fonctionnels : testez avant de commiter !
Avant de publier un commit, il est critique de vérifier que le code en est correct (tests, règles de programmation, etc.), d'autant plus s'il s'agit de séries de changements remises en forme en utilisant `git rebase -i` car une mauvaise manipulation au cours de cette opération peut introduire des régressions (même une régression uniquement présente sur des commits intermédiaires peut nuire en rendant l'usage de `git bisect` plus difficile).
En d'autres termes, testez vos changements, l'un après l'autre. Ne publiez pas des changements mal testés, vous risqueriez de faire perdre un temps précieux aux autres programmeurs et de rendre des commandes comme `git bisect` totalement inutiles.



HÉBERGEMENTS **Git en ligne : GitHub, GitLab, Bitbucket…**
Des services tels **GitHub.com, GitLab.com** et **Bitbucket.org** proposent d'héberger vos dépôts Git en ligne. Ils sont généralement gratuits pour les projets de logiciel libre. Chaque dépôt est avant tout associé à un utilisateur, et il n'est pas rare d'avoir plusieurs centaines de copies d'un dépôt pour un logiciel populaire. C'est la conséquence naturelle du modèle de développement associé : toute contribution externe est réalisée dans une copie du dépôt (`fork`) sur une branche (publique) que l'on peut ensuite soumettre pour intégration (*via* une `pull request`).
La création de dépôts s'effectue par l'interface web mais ils sont ensuite gérés directement *via* git et un dépôt distant accessible par SSH (l'utilisateur doit donc enregistrer sa clé publique SSH auprès de ces services).








N'hésitez pas à user et abuser des branches
Git a dès l'abord été conçu pour faciliter la création de branches et surtout leur fusion. Des changements peuvent ainsi rester totalement indépendants ; cela ouvre de nombreuses perspectives de travail :

- branches de correctifs **intégrables dans plusieurs versions** du logiciel ;
- **expérimentations sans risque** (il est facile de « jeter » une branche, bien moins de se débarrasser de commits entrelacés avec le développement « normal ») ;
- **branches par fonctionnalités**, pour que les autres développeurs puissent facilement tester un sous-ensemble de vos changements et vous faire des retours dessus ;
- **branches temporaires de travail collaboratif** sur une fonctionnalité avec un autre développeur avant la soumission finale, etc.

Choisissez des règles de travail et conventions communes : adoptez le même workflow
La souplesse de Git (par opposition à des systèmes de gestion de versions centralisés comme CVS, Subversion, perforce) est l'une de ses forces... mais elle peut devenir un cauchemar si tous les développeurs ne travaillent pas de la même manière. Établissez donc quelques règles dans la façon de travailler :

- utilisation d'un **dépôt central** ;
- nommage des différentes branches « officielles » ;
- procédure de contribution (par envoi de patchs par courrier électronique, par soumission de branches, par usage d'un outil de relecture...) ;
- règles pour intégrer les branches soumises.

Choisissez des outils qui s'intègrent bien avec Git
Le système de gestion de versions est la clef de voûte d'une bonne gestion de projet ; préférez ceux des autres outils classiques (gestion de tickets, relecture de code, intégration continue, etc.) qui s'intègrent bien avec Git, pour automatiser des tâches répétitives, sources d'erreurs multiples. Choisissez par exemple un système de gestion de tickets qui « comprenne » Git pour « fermer » les tickets depuis les messages de commit afin d'éviter des oublis humains. Utilisez un outil de relecture de code sachant manipuler directement un dépôt Git, pour éviter des erreurs lors de l'intégration d'une branche.

POUR ALLER PLUS LOIN **Outils de l'écosystème Git**

- **Services en ligne d'intégration continue**
  Travis CI : https://travis-ci.org
  Circle CI : https://circleci.com
- **Interfaces graphiques**
  Git Extensions (multi-plates-formes) : https://gitextensions.github.io/
  GitHub Desktop : https://desktop.github.com/
  Sourcetree : https://www.sourcetreeapp.com/
- **Gestion de patches**
  StGit gère les patches à la quilt dans Git : http://www.procode.org/stgit/
  TopGit gère les topic branches liées entre elles : https://mackyle.github.io/topgit/
- **Outils d'hébergement et de contrôle d'accès à des dépôts Git**
  GitLab : https://gitlab.com/gitlab-org/gitlab-ce
  Gitolite : http://gitolite.com/gitolite/
- **Revue de code**
  Gerrit : https://www.gerritcodereview.com/

Chez le même éditeur

Git par la pratique, D. Demaree
Mémento MySQL, R. Rimelé
Mémento HTML 5, R. Rimelé
Mémento CSS 3, R. Goetter
Mémento PHP 7 et SQL, C. Pierre de Geyer, G. Poncon
Mémento Sites web, les bonnes pratiques, E. Sloïm, L. Denis
Mémento jQuery, E. Sarrion

Attention : pour lire les exemples de lignes de code, réduisez la police de votre support au maximum.





Code éditeur : G67722
ISBN : 978-2-212-67722-5
Conception : Nord Compo
© Groupe Eyrolles, 2012.
© Éditions Eyrolles, 2019, pour la présente édition.